Exceptions and Interrupts

Instructor: Jeongseob Ahn (안정섭)

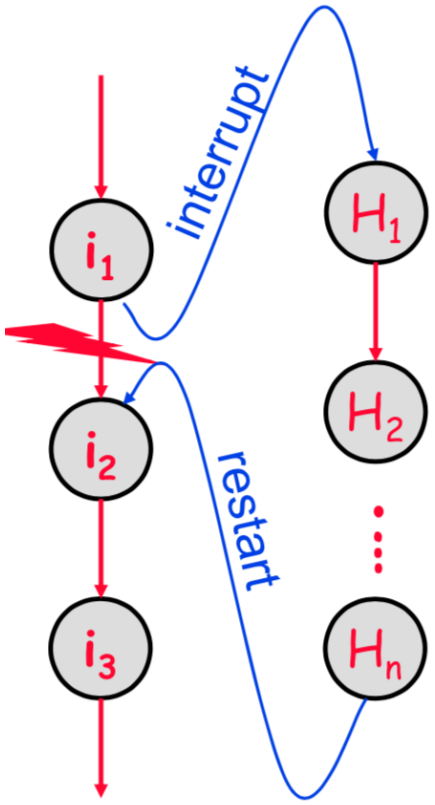
E-mail: jsahn@csl.korea.ac.kr
Hompage: https://jeongseob.github.io

CPU must prepare for ‘unplanned’ events

- A systematic way to handle exceptional conditions that are relatively rare, but must be detected and acted upon quickly
  - Instructions may fail and cannot complete
  - External I/O devices may need servicing
  - Quantum expiration in a time-shared system
- Option 1: write every program with continuous checks (a.k.a. polling) for every possible contingency
  - Acceptable for simple embedded systems (e.g., toaster)
- Option 2: write “normal” programs for the best-case scenario where nothing unusual happens
  - But, must detect exceptional conditions in HW
  - `Transparently` transfer control to an exception handler that knows how to resolve the condition and then back to your program

Interrupt Control Transfer

- An interrupt is an **unplanned** function call to a system routine (a.k.a., the interrupt handler)
- Unlike a normal function call, the interrupted thread cannot anticipate the control transfer or prepare for it in any way
- Control is later returned to the main thread at the interrupted instruction



The control transfer to the interrupt handler and back must be 100% transparent to the interrupted thread!!!

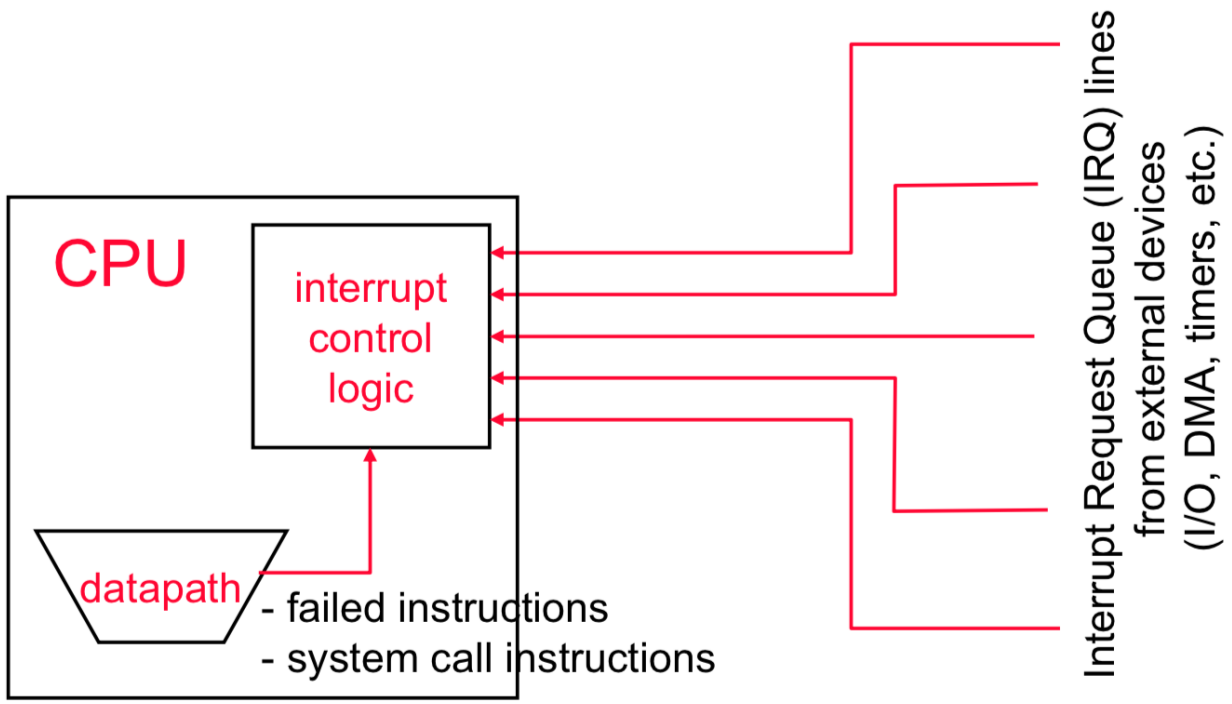
Types of Interrupts

- Synchronous Interrupts (a.k.a., **exceptions**)
  - Exceptional conditions tied to a particular instruction
    - e.g., illegal opcode, illegal operand, virtual memory management faults
  - The faulting instruction cannot be finished

  No forward-progress unless handled immediately
- Asynchronous Interrupts (a.k.a., **interrupts**)
  - External events not tied to a particular instruction
    - e.g., I/O events, timer events
  - Some flexibility on when to handle it

  Some delay allowed, but still cannot postpone forever
- System Call/Trap Instruction
  - An instruction whose only purpose is to raise an exception

Interrupt Sources



Exceptions
(a.k.a. synchronous interrupts)

Interrupts
(a.k.a. asynchronous interrupts,
or external interrupts)

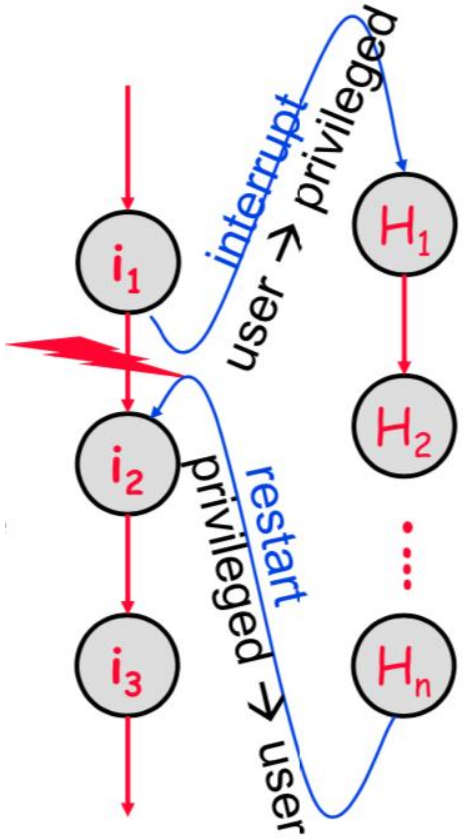
Virtualization and Protection

- Modern OS supports time-shared multiprocessing but each "user-level" process still thinks it is alone
  - Each process sees a private set of user-level architectural states that can be modified by the user-level instruction set
  - Each process cannot see or manipulate (directly) state and devices outside of this abstraction
- OS implements and manages a critical set of functionality
  - Keep low-level details out of the user-level process
    - e.g., HW device handler, thread scheduler, …
  - Protect the user-level process from each other and itself

Do you want to access/manage hard disk directly?
Do you trust your friend if he accesses the hard disk directly?

Control and Privilege Transfer

- User-level code never runs in the privileged mode
- Processor enters the privileged mode only **on interrupts**
  - User code surrenders control to a handler in the OS kernel
- The handler restores privilege level back to user mode before returning control to the user code



Implementing Interrupts

Handling Exception in RISC-V

- Save **PC** of offending (or interrupted) instruction
  - Supervisor Exception Program Counter (**SEPC**)
- Save indication of the problem
  - Supervisor Exception Cause Register (**SCAUSE**)
  - 64 bits, but most bits unused
    - Exception code field: 2 for undefined opcode, 12 for hardware malfunction, …
- Jump to handler
  - Assume at $0000\ 0000\ 1C09\ 0000_{hex}$

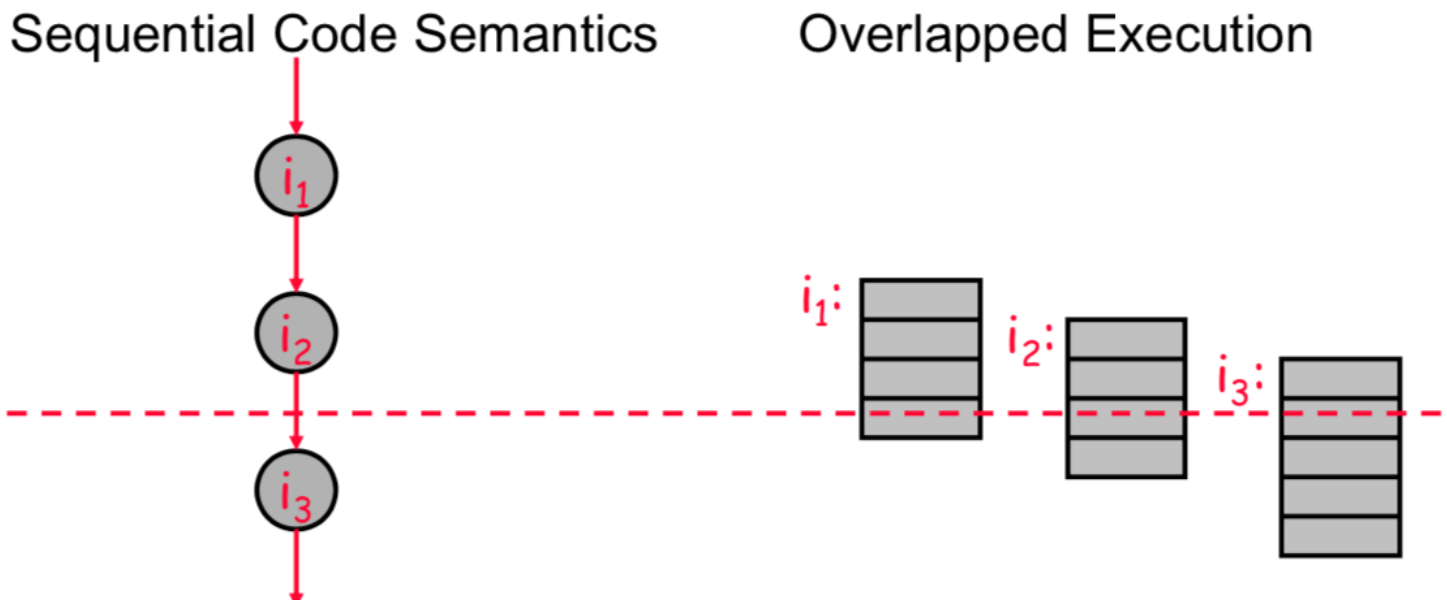
An Alternate Mechanism

- Vectored Interrupts
  - Handler address determined by the cause
- Exception vector address to be added to a vector table base register:
  - Undefined opcode: $00\ 0100\ 0000_{two}$
  - Hardware malfunction: $01\ 1000\ 0000_{two}$
  - …
- Instructions either
  - Deal with the interrupt, or
  - Jump to real handler

Handler Actions

- Read cause, and transfer to relevant handler
- Determine action required
- If restartable
  - Take corrective action
  - use SEPC to return to program
- Otherwise
  - Terminate program
  - Report error using SEPC, SCAUSE, …

“Precise” Interrupt/Exception



- A **precise interrupt** appears (to the interrupt handler) to take place exactly between two instructions
  - Older instructions finished completely
  - Younger instructions as if never happened
  - On synchronous interrupts, execution stops <u>just before</u> the faulting instruction, and resume after interrupt handling done

Exception Sources in Different Stages

- `IF`: Instruction memory address/protection fault
- `ID`
  - Illegal opcode
  - Trap to SW emulation of unimplemented instructions
  - System call instruction (an intended exception requested by SW)
- `EX`: Invalid results: overflow, divide-by-zero, ...
- `MEM`: Data memory address, protection fault
- `WB`:Nothing can stop an instruction now...

- We can associate async interrupts (I/O) with any instruction/stage we like

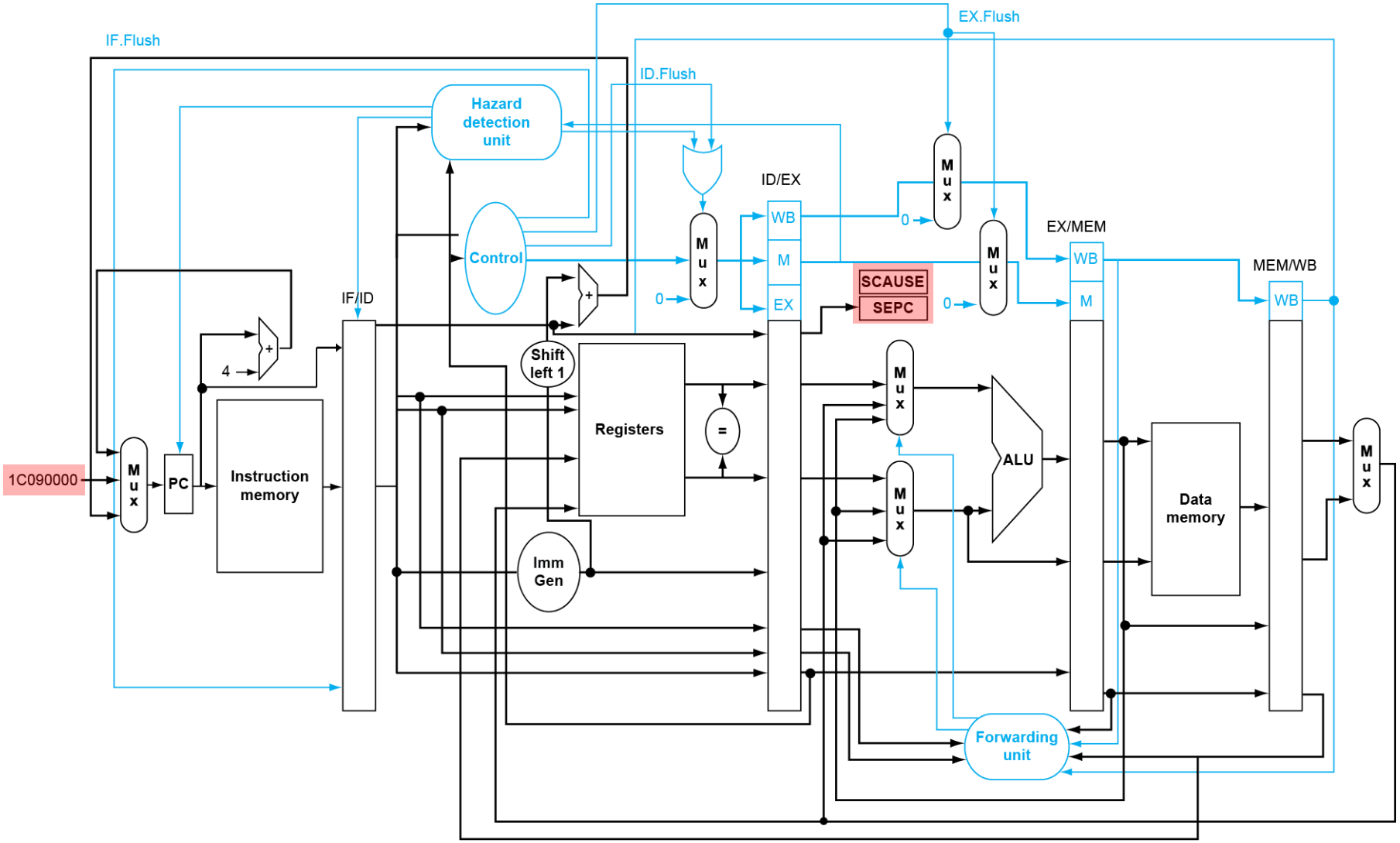
Exceptions in a Pipeline

- Another form of control hazard
- Consider malfunction on add in EX stage

  `add x1, x2, x1`

  - Prevent x1 from being clobbered
  - Complete previous instructions
  - Flush add and subsequent instructions
  - Set SEPC and SCAUSE register values
  - Transfer control to handler
- Similar to mispredicted branch
  - Use much of the same hardware

Pipeline with Exceptions



Exception Properties

- Restartable exceptions
  - Pipeline can flush the instruction
  - Handler executes, then returns to the instruction
    - Refetched and executed from scratch
- PC saved in SEPC register
  - Identifies causing instruction

Exception Example

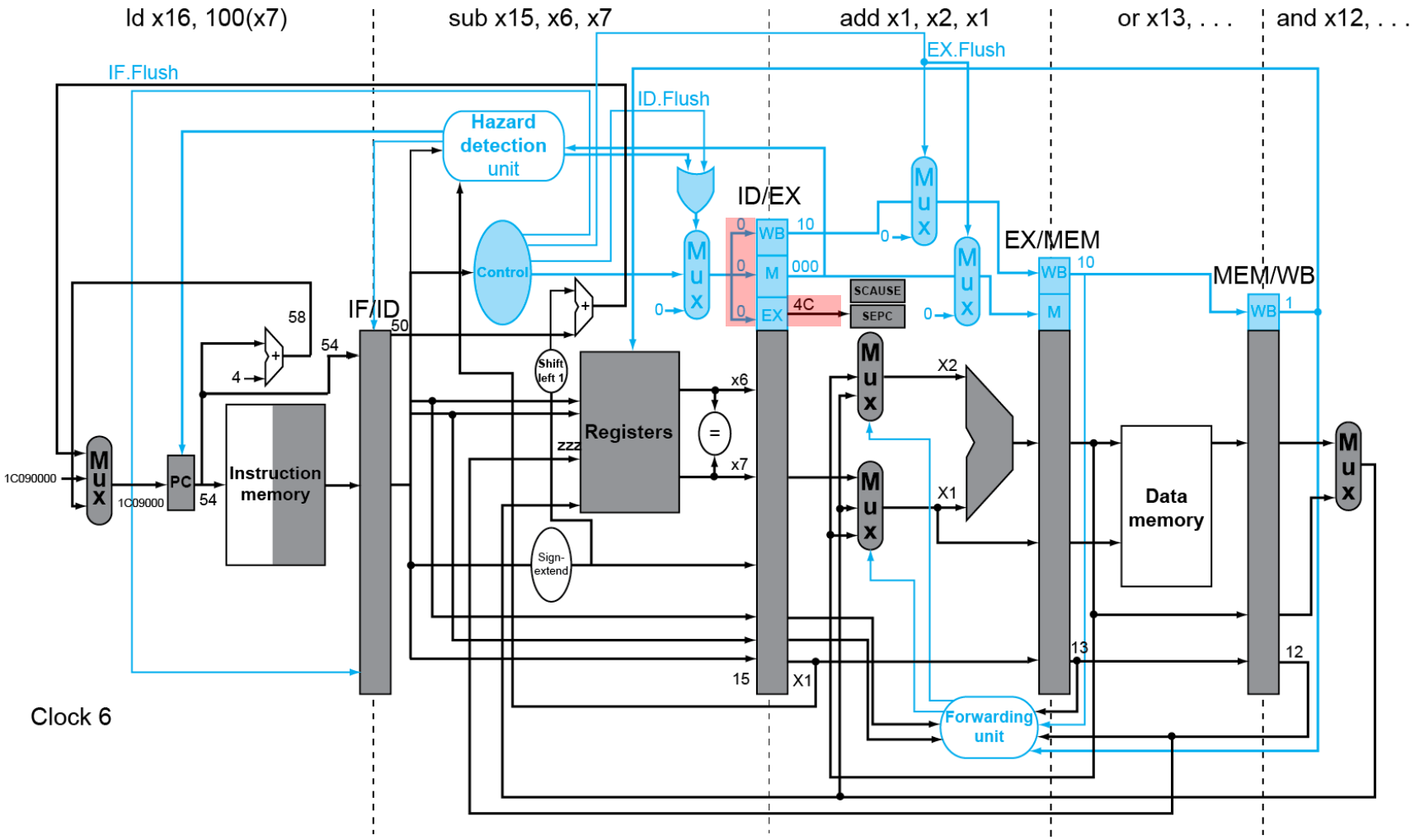
- Exception on add in

```
40    sub  x11, x2, x4
44    and  x12, x2, x5
48    orr  x13, x2, x6
4c    add  x1,  x2, x1
50    sub  x15, x6, x7
54    ld   x16, 100(x7)
…
```

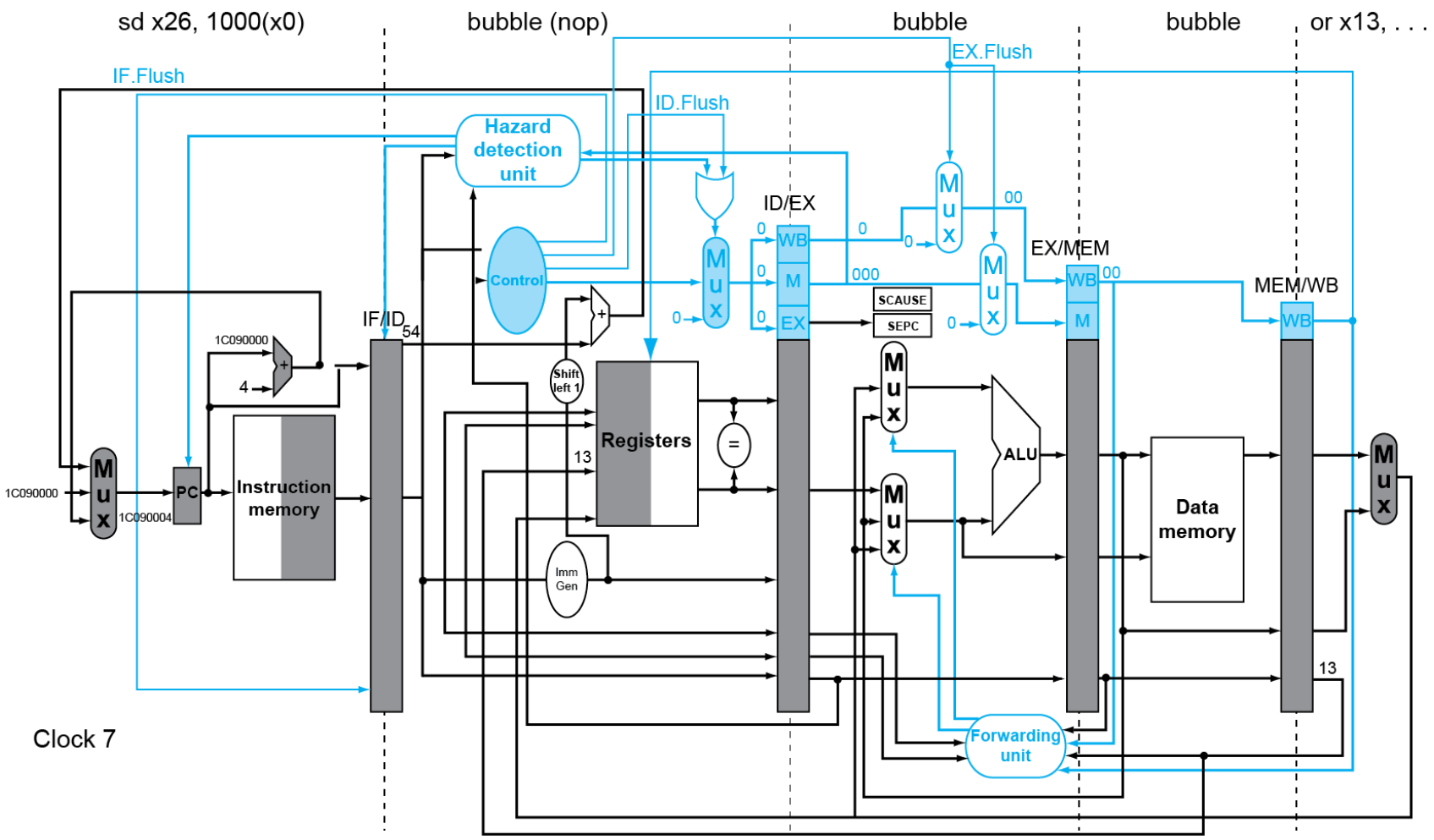
- Handler

```
1c090000    sd  x26, 1000(x10)
1c090004    sd  x27, 1008(x10)
…
```

Exception Example



Exception Example



Multiple Exceptions

- Pipelining overlaps multiple instructions
  - Could have multiple exceptions at once
- Simple approach: deal with exception from earliest instruction
  - Flush subsequent instructions
  - “Precise” exceptions
- In complex pipelines
  - Multiple instructions issued per cycle
  - Out-of-order completion
  - Maintaining precise exceptions is difficult!

Imprecise Exceptions

- Just stop pipeline and save state
  - Including exception cause(s)
- Let the handler work out
  - Which instruction(s) had exceptions
  - Which to complete or flush
    - May require “manual” completion
- Simplifies hardware, but more complex handler software
- Not feasible for complex multiple-issue out-of-order pipelines